BTP Report: Heap Reference Analysis

Gargi Bakshi and Palle Bhavana
Guide: Prof. Uday Khedker

March 3, 2026

Contents

1 Introduction **1**

2 Heap Reference Analysis Using Access Graphs **2**
2.1 Access Paths . . . 3
2.2 Access Graphs . . . 3

3 Role of Alias Information **4**
3.1 Liveness-Aware Alias Analysis . . . 5
3.2 Alias-Aware Liveness Analysis . . . 5

4 Access Path Nullification **5**
4.1 Dereferencable Access Paths . . . 5
4.1.1 Available Access Paths Analysis . . . 6
4.1.2 Anticipable Access Paths Analysis . . . 6
4.2 Nullable Access Paths Analysis . . . 6

5 Implementation Details **6**
5.1 Access Graph Operations . . . 6
5.2 Handler Class . . . 7
5.2.1 Access Graph Hashes . . . 7
5.2.2 Memoising Results . . . 8
5.3 Supporting Functions . . . 9
5.4 VASCO and SLIM . . . 10
5.4.1 VASCO Framework . . . 10
5.4.2 Using SLIM . . . 10
5.4.3 `mem2reg` Pass . . . 11

6 Modifying the Formalism **12**

7 Conclusion **15**

1 Introduction

Heap Reference Analysis (HRA) aims to summarise the properties of a program's heap data. This allows us to identify the live parts of each program's heap. Obtaining a distinction between reachable and live objects in a program's heap using such analysis has several applications, primarily improved garbage collection, which typically only cleans up the unreachable parts of the memory.

Our B.Tech. project is implementing HRA in VASCO (VAlue Sensitive COntexts) framework using SLIM (Simplified LLVM IR Modelling), an abstraction layer over LLVM with simplified data structures and function calls. Our work began with developing an understanding of LLVM and how passes in LLVM work. We then learned about access graphs, which are used to perform the heap reference analysis, and the operations and flow functions used in this static analysis. This was followed by exploring how to interpret various instructions and operands in SLIM IR to generate and modify access graphs, and

implementing access graph operations and other helper functions that would be used in the flow functions for the analysis.

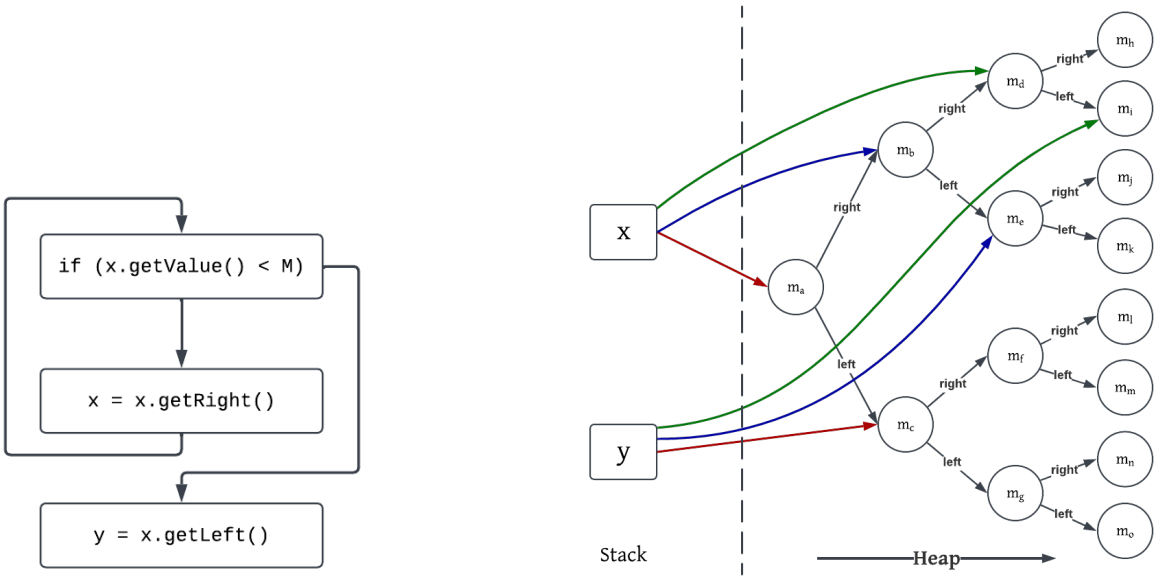
2 Heap Reference Analysis Using Access Graphs

HRA involves analysing the program structure to identify the properties of the heap data of that program. *Heap Reference Analysis Using Access Graphs* [1] details a static analysis to obtain bounded representations of these properties using the storeless memory abstraction called access graphs. This allows us to identify patterns in heap accesses made in a program without relying on the actual addresses of the dynamic memory being allocated, which will only happen at runtime.

If one were to distinguish live data that the program can access during runtime from all reachable data in the memory graph, this would lead to improved garbage collection. Collecting reachable but not live objects can be done by setting the appropriate references to *null*. This would result in improved garbage collection, an idea known as *Cedar Mesa Folk Wisdom*.

We consider a snippet of a Java program given below in which the addresses being held by variables x and y depend on the number of times the while loop is executed.

```
while (x.getValue() < MAX) {
    x = x.getRight();
}
y = x.getLeft();
```



(a) CFG for the program fragment

(b) the corresponding memory graph

Figure 1: A motivating example

Figure 1a shows the control flow graph for the program snippet given earlier in which the heap accesses depend on how many times the while loop is executed. In such cases, the heap data accessed is not a program invariant. Figure 1b shows how different parts of the heap memory are accessed depending on the number of iterations of the while loop. The red, blue and green arrows indicate the memory locations pointed to by x and y upon 0, 1, and 2 iterations of the while loop, respectively. Static analysis, therefore, cannot identify the memory locations in a store-based model like a *memory graph* which specifies the memory locations being accessed by a series of dereferences.

2.1 Access Paths

The edges (links) in the memory graph are labelled by field names. An *access path* is a root variable name followed by a sequence of zero or more field names and is denoted by $\rho_x = x \rightarrow f_1 \rightarrow f_2 \rightarrow \cdots \rightarrow f_k$. A *simple access path* consists of just the root variable name. An *empty access graph* is denoted by $\mathcal{E}$.

For example, considering the variable x from Figure 1, an access path $\rho_x = x \rightarrow left$ can correspond to memory locations m_c, m_e or m_i, depending on whether the while loop is executed 0, 1 or 2 times.

Here are some terminologies related to access paths:

- *Frontier*: The last field name in an access path ρ is called its frontier and is denoted by $Frontier(\rho)$. The frontier of a simple access path is the root variable name.
- *Base*: The access path corresponding to the longest sequence of names in ρ, excluding its frontier, is called its base and is denoted by $Base(\rho)$. The base of a simple access path is $\mathcal{E}$.
- *Target*: The object reached by traversing an access path ρ is called the target of the access path and is denoted by $Target(\rho)$.

For a link l to be live, there must be at least one access path from some root variable to l such that every link in this path is live. An access path is defined as live at p if the link corresponding to its frontier is live along some path starting at p. For example, in the CFG shown in Figure 2, the set of live access paths at the entry of 1 is

$$S = \{x, x \rightarrow n, x \rightarrow n \rightarrow n, \cdots\} \tag{1}$$

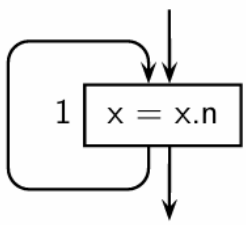


Figure 2: A part of a program's control flow graph

We define Gen_n and $Kill_n$ for each instruction n involving field accesses to get the access paths for objects on the heap. The data flow equations for explicit liveness analysis are then defined using In_n and Out_n as follows:

$$In_n = (Out_n - Kill_n(Out_n)) \cup Gen_n(Out_n) \tag{2}$$

$$Out_n = \begin{cases} BI, & \text{if } n \text{ is } End \\ \bigcup\limits_{s \in succ(n)} In_s, & \text{otherwise} \end{cases} \tag{3}$$

2.2 Access Graphs

In the presence of loops, as in Figure 2, the number of access paths is infinite. Therefore, instead of explicitly computing the set of access paths, we use a bounded representation called *access graphs*.

An access graph G_v is a directed graph $< n_0, N, E >$ representing a set of access paths starting from a root variable v. N is the set of nodes, $n_0 \in N_F$ is the entry node with no in-edges and E is the set of edges. Every path in the graph represents an access path. The *empty graph* $\mathcal{E}_G$ has no nodes or edges and does not accept any access path.

The entry node of an access graph is labelled using the root variable name. The non-entry nodes have unique labels $< f, i >$, where f is the field name and i is the instruction ID (following the SLIM notation).

Figure 3 shows the access graph at the start of basic block 1 in Figure 2. This graph represents the infinite set of access paths given by equation 1.

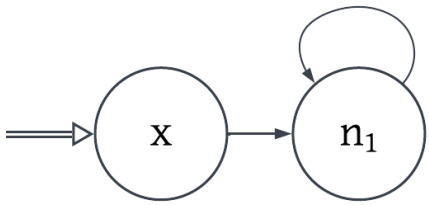


Figure 3: Access graph for Figure 2

We define Gen_n and $Kill_n$ for access graphs using some operations:

- *Union*
- *Path Removal*
- *Factorization*
- *Extension*

Remainder Graphs

Given a node $m \in N \setminus \{n_0\}$ for access graph G, the remainder graph of G is the subgraph rooted at m and is denoted by $RG(G, m)$. Its entry node corresponds to a field name instead of a variable.

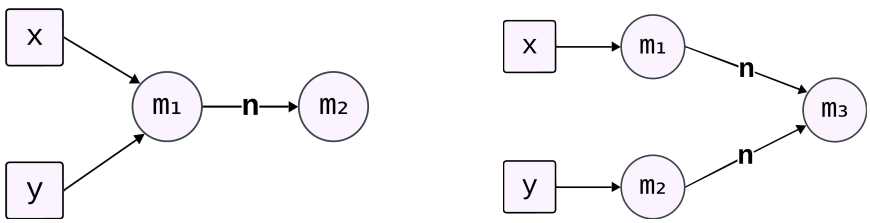
Remainder graphs are used in the factorization operation to obtain the subgraphs at the end of an access path, and in the extension operation to append subgraphs to an access graph.

3 Role of Alias Information

Standard liveness analysis does not factor in alias information. This may lead to imprecision and unsoundness.

Two pointer expressions are *aliases* if they point to the same memory location. This is represented in the memory graph by multiple paths reaching a node. There are two types of aliases for pointer expressions:

- *Link aliases*: The access paths corresponding to the two pointer expressions share a common link in the memory graph.
- *Node aliases*: The access paths corresponding to the two (aliased) expressions share no link.



(a) `x.n` and `y.n` are link aliases (b) `x.n` and `y.n` are node aliases

Figure 4: Link vs node aliases for pointers to a struct `S` with `struct S* n` as a field

If an access path is live, all its link aliases are live too. However, its node aliases need not be live.

We need to take this alias information into account and modify the liveness analysis in the following ways:

- If an access path is killed, all its must-link aliases should also be killed. This avoids spurious liveness.
- When the transfer of liveness happens through factorisation and extension, all access paths that are may-link aliases of the LHS should be factorised. This gives us some implicitly live access paths whose liveness depends on the alias information at a node and shouldn't be propagated backwards.

Similar to LFCPA (liveness-based points-to analysis), where liveness information and points-to information depend on each other, for heap analysis, liveness information and alias information depend on each other. Therefore, we perform alternating rounds of alias-aware liveness analysis and liveness-aware alias analysis till we reach convergence. The alias analysis is performed on access paths to record node aliases (since this information can derive link aliases), and liveness information is recorded using access graphs.

Access graphs with cycles can represent an infinite number of access paths. Therefore, the alias information that is restricted using the liveness information represented by the access graphs may record an infinite number of aliases. However, this is not the case. Since the access graphs are finite, there can only be a finite number of node aliases, and the link aliases using cycles in the graph can be derived from these acyclic node aliases. Therefore, we record only the node aliases corresponding to acyclic paths, which ensures the finiteness of alias information.

3.1 Liveness-Aware Alias Analysis

Alias information is recorded as a set of pairs of access paths (ρ, ρ'). Although aliasing is a symmetric relation, the asymmetry used in this format allows us to record that the frontier of ρ was defined by the assignment that created the alias.

We require that the access paths used for computing aliases be acyclic to ensure finiteness. Also, since the analysis is liveness-aware, at least one path in a set of aliased access paths must be explicitly live.

3.2 Alias-Aware Liveness Analysis

We need to modify the default liveness analysis to also consider aliases. If the path removal operation is being used to kill the liveness of an access path, we must also kill the liveness of all its must-link aliases. Moreover, when we perform factorisation during the transfer of liveness from the LHS to the RHS, we must factorise using all the link aliases of the LHS too, and extend all the remainder graphs to the RHS's access path. These changes require modifying the path removal and factorisation operation to take a set of access paths as input instead of a single access path.

4 Access Path Nullification

In the earlier sections, we used access graphs to represent the live parts of the heap. Our goal is to identify program points where we can insert nullification statements at the entry of each node for pointer expressions. While nullifying these expressions, we have to consider two factors:

- **Soundness**: No access path representing the pointer expression should be live, either implicitly or explicitly. We ensured this by factoring in alias analysis, as described in Section 3. Moreover, every sub-expression of that pointer expression must be live. Such paths are called *dereferencable access paths*. We use available and anticipable access path analyses to identify such paths.
- **Precision**: We want to nullify a pointer expression as early as possible in the program, and avoid nullifying it multiple times. Also, we should not nullify an access path if one of its proper prefixes can be nullified, since not doing so introduces an unnecessary statement. We also shouldn't nullify must-link aliases of already nullified paths. These requirements are handled by the *nullable access paths analysis*, which gives us the set of nullified access paths in a forward traversal.

4.1 Dereferencable Access Paths

An access path is called *dereferencable* if a target location exists in the heap for each of its non-empty prefixes for all executions of a program point. Such paths represent non-null pointer expressions.

We use available and anticipable access path analyses to get the set of dereferencable access paths at each program point. Dereferencability is a must-property, and we under-approximate the set of dereferencable access paths.

$$DereferAPin_n = AvAPin_n \cup AnAPin_n$$

4.1.1 Available Access Paths Analysis

An access path is available at a program point if a target location for it exists in the heap along all paths reaching that program point. The set of available access paths is prefix-closed. This is a forward analysis similar to available expressions analysis, with intersection as the confluence and ϕ as the initialisation.

We generate the liveness of all non-empty proper prefixes of the access paths for the LHS and RHS expressions. The access path for the LHS is made conditionally available depending on whether the access path for the RHS is available or not. An available access path is killed if it is redefined.

4.1.2 Anticipable Access Paths Analysis

An access path is available at a program point if its frontier is dereferenced along all control flows starting at that point. The set of anticipable access paths is prefix-closed. This is a backward analysis similar to anticipable expressions analysis, with intersection as the confluence and ϕ as the initialisation.

The transfer of anticipable access paths is done in the same way as liveness analysis. The killing is done using the alias-aware path removal operation.

4.2 Nullable Access Paths Analysis

This analysis gives us the set of access paths nullified at the start of each basic block. It is a forward analysis with intersection as the confluence.

The dataflow value $NullAPin_n$ represents the access paths that have already been nullified before the current block n.

The paths that have to be nullified at the start of n are represented by $NullAPgen_n$, and are computed using a function `findNullable`, which takes as input the dereferencable paths, already nullified paths, the live access graph, and alias information. The function calls for each pointer expression p a procedure `nullify` which checks if the pointer is, or can be, nullified. If not, the `nullify` procedure is called for all expressions $p \rightarrow f$ where f is a valid field for p.

We incorporate the information of the newly nullified blocks while computing $NullAPout_n$, and remove the paths that have been redefined from the set of nullified access paths.

5 Implementation Details

This analysis will be done in the VASCO (VAlue Sensitive COntexts) framework, which uses SLIM (Simplified LLVM IR Modelling) representation. As part of the implementation, we decided on the data structures for the data flow analysis, implemented supporting functions such as the access graph operations, created frameworks to store the data flow values efficiently, and analysed SLIM IR for various programs to decide how to process instructions to construct access graphs.

5.1 Access Graph Operations

The four access graph operations are implemented in C++ and do not require any VASCO or SLIM support. We created classes for access paths, access graphs, and remainder graphs.

An access graph node is either a root variable name representing the start of an access path or a field name with a subscript, which is its basic block ID. For constructing graphs for which the nodes are created using SLIM operands, we need to store the instruction ID instead of the basic block ID since an access graph will be made at each relevant instruction's in and out. Moreover, we can have pointer expressions like $x \rightarrow f \rightarrow f$ in one line. Since we cannot have the same subscript (instruction ID) for two field nodes, we need an additional index to store this. All this information can make an access graph node very bulky and increase the runtime for operations. Therefore, we create a `NodeMapper` helper class which assigns a unique number to each possible node in access graphs, allowing us to have integers as nodes in the access and remainder graphs.

The adjacency list in the access and remainder graphs is made using `unordered_maps` because of faster lookup, insertion and deletion. The average time complexity for these operations is $\mathcal{O}(1)$ in `unordered_maps`, as opposed to the $\mathcal{O}(\log n)$ complexity of `maps`. Using an `unordered_set` to store the neighbours of a graph ensures that there are no duplicates. Since we don't need to maintain an order of the neighbours, an `unordered_set` is better than a `set`, in which the elements are ordered. Hash collisions are handled internally using methods like collision chaining or hash buckets. Therefore, collisions are safe but may lead to higher look-up times, the worst case being $\mathcal{O}(n)$ complexity.

The implementation of the union and extension operations is straightforward and does not require complicated updates to the adjacency matrix. For the factorisation operation, the first step is to identify all the paths in the graphs that correspond to the access path given as an argument. Once we get the set of all such paths, the result is obtained as the set of all subgraphs, ie remainder graphs, rooted at the last node (called *target node*) of each such path.

The algorithm for the path removal function is defined in the chapter *Liveness Analysis for Heap* in the book. It works by first identifying the target edges in the access graph, which correspond to the last edge in the access path ρ to be removed, such that no edge in the access graph is repeated while traversing it along ρ. All the edges leading up to the target edges are coloured *green*, since these belong to a proper prefix of ρ and must not be removed. Every other edge is unexplored, and it must be determined whether to keep or remove that edge. This is done in two stages - marking all potentially removable edges *red*, and then unmarking the edges that need to be retained. In the first stage, we start by marking all target edges *red*, and then recursively marking the successors as well, provided they're not a non-target edge along some traversal of ρ. In the second stage, we unmark those *red* edges that have an incoming *white* (unmarked) edge, since this indicates that the marked edge was also being used for representing an unrelated access path and therefore cannot be removed. After these two stages, the red edges are deleted from the graph, and the connected component of the access graph containing the start node is returned as the result of the path removal operation.

5.2 Handler Class

The Handler class is used to store access graphs efficiently and memoise graph operations' results wherever possible. This is helpful since we may need the same access graph at multiple program points or across multiple iterations of the analysis. Recomputing the graph in such cases can be time-consuming and memory-inefficient. Our goal is to avoid redundant computation and not create multiple copies of the same graph.

The Handler stores a mapping from the hash value of an access graph to a pointer to the graph object for each access graph created in the analysis. These hashes are used to represent the graphs in a data structure which memoises graph operation results.

5.2.1 Access Graph Hashes

We created a custom hash function for access graphs, which is used by the Handler to memoise results efficiently. The hash is computed as a combination of hashes of each edge in the graph, with the edges being processed in lexicographically increasing order. This means that edge (n_1, n_2) is processed before edge (n_1', n_2') if $n_1 < n_1'$, or $n_1 = n_1'$ and $n_2 < n_2'$. Maintaining an order while computing hashes is necessary to avoid cases where two graphs have the same structure, but have had the edges added in different sequences. Moreover, using only edges and not nodes in the adjacency matrix avoids cases where structurally equal graphs have different hashes because one of them had a node with an empty set of neighbours while the other did not have that node in the list at all.

While we ensure that equivalent graphs don't have different hashes, we also need to handle hash collisions. We observed by generating multiple random graphs of certain sizes that the hash function works extremely well. However, upon checking for graphs of large sizes ($\sim 10,000$ nodes), we observed some graph collisions. Since there is no upper bound on the size of a program, and consequently the size of the access graphs, we need a mechanism for handling collisions.

Whenever a new access graph G is created, we first add it to the hash map in the Handler. We have three cases:

- If the G's hash is already present in the hash map, and G is equal to the hash's value graph in the map, we utilise the already stored pointer for subsequent uses of that graph instead of also storing

the new pointer. This ensures that multiple copies of the same graph aren't stored.

- If the G's hash is already present in the hash map, and G is not equal to the hash's value graph in the map, we have a hash collision. The method to handle such collisions is described below.
- If G's hash is not present in the hash map, we create a new (*hash, pointer*) entry.

Hash collisions can be handled in two ways:

- The first way is to modify the hash map so that the value is a list of graph pointers instead of a single pointer, so that a unique pointer can be stored for each graph with the same hash.
- The second way is to retain the default mapping from hash to pointer, and model a hash collision using the existing structure. We do this by using a null pointer as the value of a hash for which multiple graphs have been found.

The first method gives us a way of ensuring that we store only one pointer for each unique graph. The second method has a drawback in this aspect since graphs corresponding to a collision won't have any information stored about them so we may end up with multiple copies of such graphs. However, we observed that the hash function works very well and collisions are extremely rare. Therefore, adding the additional memory for each entry in the hash map (for the first method) may end up being memory inefficient. Pointers take up 8 bytes and vectors take 24 bytes on the stack, plus the required amount of heap data to store all the elements in the vector. This is an unnecessary memory overhead since collisions are an edge case. Taking these considerations into account, the current implementation uses the second approach. This can be modified if experiments on actual programs show more frequent hash collisions.

In summary the current implementation has the following features for the hash map:

- *Hash not present*: No graph found so far with the given hash
- *Hash present and non-null pointer*: Unique graph present for the given hash, as of now
- *Hash present and null pointer*: Multiple graphs present for the given hash

5.2.2 Memoising Results

An important function of the Handler is retaining results of the operations performed so far, so that if the same operations need to be performed again, we can directly return the result without any recomputation. The results are memoised using a directed graph, in which the nodes are hashes of access graphs, and edges carry information about the operations that are performed on the nodes from which they originate.

We have two types of edges in the universal graph - one from the union operation and one from the path removal operation. Since factorization returns a set of remainder graphs, we don't use store results from it in the universal graph. The extension operation does return an access graph but requires a set of remainder graphs and nodes as input which cannot be used for labelling edges. Therefore, we create edges in the universal graph using the other two operations as follows:

- *Union edges*: In the operation $G = G_1 \uplus G_2$, we make edges from G_1 and G_2 to G. The edge $G_1 \rightarrow G$ will be labelled $(G_2, \text{'u'})$, indicating that performing the union operation on G_1 with G_2 results in G. Similarly, The edge from G_2 to G will be labelled as $(G_1, \text{'u'})$. Figure 5 shows the subgraph obtained from this union operation.
- *Path Removal edges*: In the operation $G_2 = G_1 \ominus \rho$, we make an edge labelled $(\rho, \text{'p'})$ from G_1 to G_2, indicating that G_2 is the result of removing access path ρ from G_1.

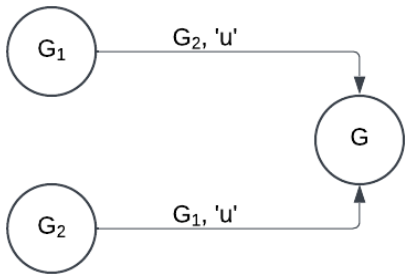


Figure 5: Memoising the result $G = G_1 \uplus G_2$

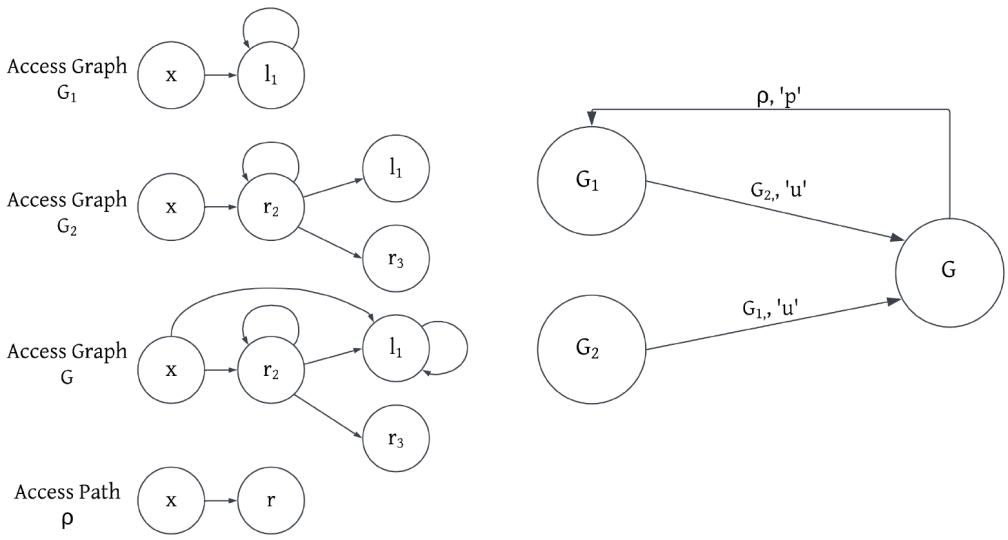


Figure 6: Memoised results for both union and path removal

Figure 6 shows the memoised results graph with both union and path removal edges. The union edges are obtained through $G = G_1 \uplus G_2$ and the path removal edge is obtained through $G_1 = G \ominus \rho$. After these edges are added, if we again want the result of these operations, the implementation will first check the universal graph for the existence of the required edges from the nodes and return the result directly without performing the operations again.

Since we use hashes in this graph, it is important that none of the hashes have collisions. In the hash map, collisions are handled by setting the hash's value to null, as mentioned in Section 5.2.1. Upon encountering a collision, we also modify the memoised results graph by removing all nodes and edges corresponding to the hash for which a collision was found. This ensures that the information stored in the graph is associated only with the graphs that have unique hashes.

5.3 Supporting Functions

We also created some supporting functions to make liveness-aware alias analysis easier:

- *AcyclicPaths*: It takes an access graph and a set of access paths as input and retains only those access paths that are acyclic in the graph. It uses the target edges and target node sets defined for liveness analysis.
- *Relevant*: It is a predicate that takes an access graph and a set of access paths as inputs and checks that at least one path is explicitly live in the access graph

- *AccessPaths*: It takes an access graph as input and returns all the acyclic access paths it represents. This is done using DFS and adding all paths visited during the traversal. To ensure that all paths are visited, nodes are marked while traversing one path and then unmarked so that they can be explored by other paths in the future.

5.4 VASCO and SLIM

VASCO (VAlue Sensitive COntexts) framework is used to implement data flow analyses using the SLIM intermediate representation. SLIM (Simplified LLVM IR Modelling) is an abstraction over LLVM with simplified API calls and more readable IR. It processes a `.ll` file (human readable LLVM IR), and uses it to populate SLIM data structures such as SLIM operands, Instructions, etc.

5.4.1 VASCO Framework

VASCO framework provides a set of functions that we have to populate for our custom analysis by defining what our data flow values should be. The main function to be implemented for a backward analysis is `computeInFromOut`, which will take the data flow value at the *out* of that instruction block (a `BaseInstruction`) and analyse the instruction to obtain the data flow value at the *in*.

Some other important functions are `performMeet` at branches (union), `getInitialisationValue`, `getBoundaryInformation`, etc. Currently, VASCO is undergoing a migration, so we have not worked directly on it. Instead, we attempted to decide how the analysis should be implemented by looking into how current analyses are implemented in it.

For liveness analysis, the data flow value would be a set of access graphs. We would create a global instance of the Handler to handle all the graphs created in the `computeInFromOut` and `performMeet` functions.

5.4.2 Using SLIM

The primary task for implementing the analysis in VASCO is understanding how to identify relevant SLIM instructions and process them to create access graphs. We constructed several examples to come up with a procedure for analysing these instructions, which led to some unexpected roadblocks and interesting insights.

We first need to identify pointers to structs in an instruction, and then construct or modify an access graph by processing the fields accessed in an instruction. The `CallInstruction` is used for dynamic memory allocation, by calling calloc or malloc (for C), to initialise pointers. For example, the following line in C for some `struct S`:

```
struct S * first = (struct S *)calloc(1, sizeof(struct S));
```

corresponds to the following two instructions in SLIM:

```
[src: 12] call_main = call calloc(1, 16)
[12] i_main = (%struct.S*) call_main
```

More generally, we want to identify all SLIM operands corresponding to pointers to structs. This information can be obtained using the LLVM type of the operand, accessed using the `getType()` function. The `typeID` should be `PointerTyID`. Once we identify pointers, we can check that they are pointers to structs using the `getContainedType(0)` function of `llvm::type`, which should give us `StructTyID` as the `typeID`. Once we identify pointers to structs, we can get the underlying struct in the following way:

```
const llvm::Type * containedType = llvmType->getContainedType(0);
const llvm::StructType * structType
                            = llvm::dyn_cast<llvm::StructType>(containedType);
```

Here, `llvmType` is obtained using `getType()` on the SLIM operand. Once we have `structType`, information such as its name and fields can be accessed as follows:

```
string structName = structType->getName().str();
for (unsigned i = 0; i < structType->getNumElements(); ++i) {
    const llvm::Type *fieldType = structType->getElementType(i);
}
```

This information can help us identify the structs of interest, namely those that have a pointer to a struct (of the same type) as a field.

Another highly useful member function of SLIM operands is `getRefersToSourceValues()`. This function returns a set of SLIM operands that could be the source code variables associated with a given SLIM operand. This information helps handle the SLIM operands that represent some non-root pointer expression, such as $x \to f \to f$. For the operand representing the given expression, getRefersToSourceValues() would return the SLIM operand storing x. The level of indirection, ie, the number of accesses from a root variable, does not affect the information stored in the refers-set of the SLIM operand.

Next, we want to formalise how to process instructions that use these operands. We can have *uses* of such pointers through statements like return, boolean expressions for equality checks, non-`struct*` type field accesses, etc, or *assignments* that would cause a transfer (factorisation + extension) in the access graphs. In the source code, each such statement has a fixed format for a pointer expression - a root variable name followed by a valid sequence of field accesses. However, this doesn't hold in SLIM because of an optimisation, which is described in Section 5.4.3.

5.4.3 `mem2reg` Pass

The `mem2reg` pass is used while generating SLIM IR to reduce the number of intermediate SLIM operands created for storing intermediate results and loading root variables. This pass is essential for maintaining the scalability of various analyses for large programs.

Consider the following two instructions in a program on lines 12 and 13, in which `a`, `b`, and `c` are pointers to a struct with `left` and `right` fields.

Listing 1: Example source code

```
b = a->left;
c = b->right->right;
```

Listing (2) Without mem2reg

```
[5][12] i_foo = a.addr_foo
[6][12] <left_foo, 1> = <i_foo[0][0], 0>
[7][12] i1_foo = *left_foo
[8][12] b.addr_foo = i1_foo
[9][13] i2_foo = b.addr_foo
[10][13] <right_foo, 1> = <i2_foo[0][1], 0>
[11][13] i3_foo = *right_foo
[12][13] <right1_foo, 1> = <i3_foo[0][1], 0>
[13][13] i4_foo = *right1_foo
[14][13] c_foo = i4_foo
```

Listing (3) With mem2reg

```
[0][12] <left_foo, 1> = <a_foo[0][0], 0>
[1][12] i1_foo = *left_foo
[2][13] <right_foo, 1> = <i1_foo[0][1], 0>
[3][13] i3_foo = *right_foo
[4][13] <right1_foo, 1> = <i3_foo[0][1], 0>
[5][13] i4_foo = *right1_foo
```

Figure 7: SLIM IR for the code given above in 1

Figure 7 shows us the instructions generated for the source code given in 1, in which the IR is generated without and with the `mem2reg` pass. We notice that this pass eliminates a lot of unnecessary intermediate operands.

For line 12, without `mem2reg`, we have four instructions in which `a` is loaded into an operand, the pointer expression `a->left` is computed using a pair of {*get element pointer*, *load*} instructions, and the result is stored in `b`. A similar flow is followed for line 13.

However, with `mem2reg`, line 12 has only two instructions corresponding to itself - *get element pointer* and *load*. This pair of instructions is simply used to compute the pointer expression represented by the RHS. We don't have any instruction in which the operand storing the result of the expression is assigned to the variable `b`. Instead, we see in the instructions for line 13 that this result operand `i1_foo` is directly used instead of `b_foo`. This highlights an interesting optimisation done by `mem2reg` - *store* instructions for assigning pointer expressions to root variables are eliminated, and whenever those variables are used subsequently, the intermediate SLIM operand is directly used in place of the root variables.

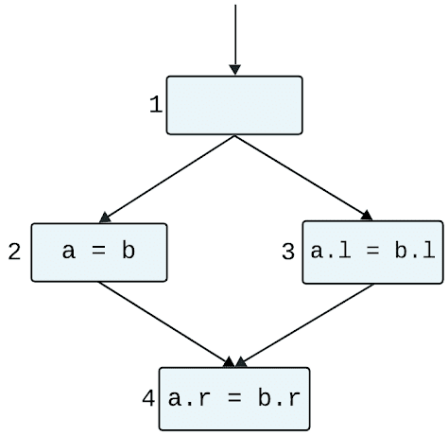


Figure 8: CFG of a program fragment which uses a `phi` instruction

Figure 8 shows the control flow graph of a program with an if-else statement, which leads to branching. In the absence of `mem2reg`, the IR would contain instructions to load each root variable into an operand, evaluate the pointer expressions and store the result back into the variable, for each instruction.

However, `mem2reg` makes an optimisation in the form of *phi* instructions. If, during execution, the path $1 \rightarrow 3 \rightarrow 4$ is taken by the program, then the SLIM operand for `a` at point 4 would be the same as the SLIM operand for `a` at point 3. If the path $1 \rightarrow 2 \rightarrow 4$ is taken, then the assignment statement at 2 would cause the SLIM operand for `a` at point 4 to be the SLIM operand for `b` at point 2. Therefore, the operand to be used depends on runtime. To handle this without generating additional instructions at points 2 and 3 (for explicitly storing the results in `a`), a *phi* instruction is used:

```
[16]a.addr.0_foo = phi(b_foo, a_foo)
```

This instruction states that depending on the control flow taken by the program during execution, `a.addr.0_foo` would be either `b_foo` or `a_foo`.

6 Modifying the Formalism

In Section 5.4.3, we saw the optimisations done by `mem2reg`. We need to specify how access graphs must be generated and modified in such cases.

For *phi* instructions, the access graph generation can be handled by considering both the operands that are the arguments for the *phi* function for the update. Therefore, for the program fragment given in Figure 8, at point 4, we would generate the liveness of both `a_foo` and `b_foo` to handle the *phi* function for `a.addr.0_foo`.

We also noticed that source code assignments to root variable pointers aren't recorded as a *store* instruction. Because of this, an assignment statement may have possibly 0 instructions corresponding to

it (for example, point 2 in Figure 8). Although this optimisation causes us to deviate from our method of access graph generation for source code, it doesn't affect the liveness analysis since we end up working with an equivalent representation of our program, which causes a transfer in liveness to happen one step in advance.

The second possibility for assignment statements is that the IR may only have instructions to evaluate the pointer expression, as given in the code segment for lines 12 and 13 for a program in 1 without an actual "assignment". In this case, we treat that instruction as a *use* statement rather than an assignment (since we cannot infer through the IR that its source code corresponds to an assignment statement) and generate access graphs accordingly.

One optimisation, however, requires us to modify our definition of access graphs to generate them at each program point. In 1, the IR with `mem2reg` has 4 instructions for line 13. The first is a *get element pointer* instruction which uses the operand `i1_foo`, which does not correspond to a root variable but stores the evaluated pointer expression `a->left`. Because of this, during the backward analysis, when we want to construct the graphs for line 13, we don't have a valid start node for the access graph for the RHS. This calls for *intermediate access graphs*, whose start nodes are evaluated pointer expressions, rather than source code variables. We illustrate how to construct and modify these using the portion of a program's CFG given in Figure 9, from the code in 1.

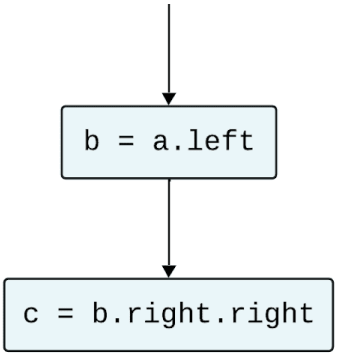


Figure 9: CFG of the program in 1

```
[0][12] <left_foo, 1> = <a_foo[0][0], 0>
[1][12] i1_foo = *left_foo
[2][13] <right_foo, 1> = <i1_foo[0][1], 0>
[3][13] i3_foo = *right_foo
[4][13] <right1_foo, 1> = <i3_foo[0][1], 0>
[5][13] i4_foo = *right1_foo
```

use a.left

use i1.right.right

Figure 10: We interpret the IR generated for the two lines in 1 as *use* statements to generate the liveness of the RHS - line 12 becomes *use* `a.left`, and line 13 becomes *use* `i1.right.right`, where `i1` is the operand storing the pointer expression `a.left`

Figures 11, 12, 13, and 14 outline how the access graphs for the CFG in Figure 9 are generated at the entry of both basic blocks. The main idea is to generate access graphs for all statements per the usual procedure, by considering the intermediate result operands as root variables. As we traverse backwards, we would eventually reach the point where these result operands are defined. At that point, we merge the intermediate access graphs to their *parent*, so that we have an access graph with a root variable as the start node. The intermediate access graphs are essentially sub-graphs that have to be attached to their parent at a later stage.

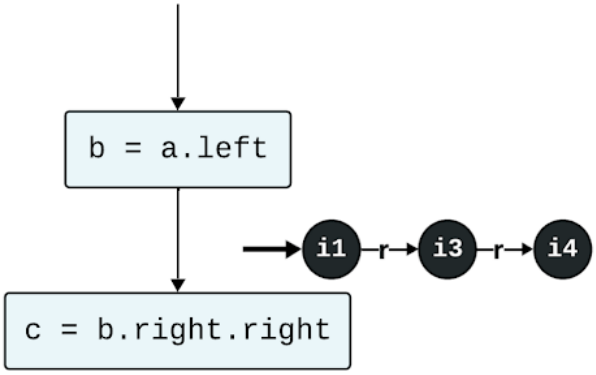


Figure 11: Here, we generate the liveness of the expression of the RHS of the lower block (line 13 originally) because of our interpretation of the assignment as *use* `i1.right.right`, to get an *intermediate access graph*. `i1` is a non-root variable which stores a pointer expression

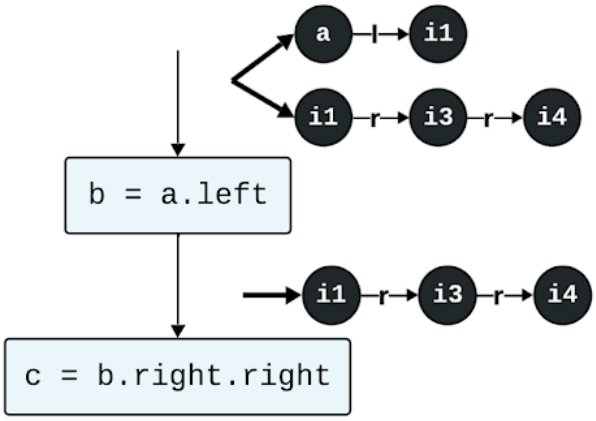


Figure 12: Now, we come to the entry of the upper block (line 12). Here, we have interpreted the assignment as *use* `a.left` and therefore generate the liveness of the RHS

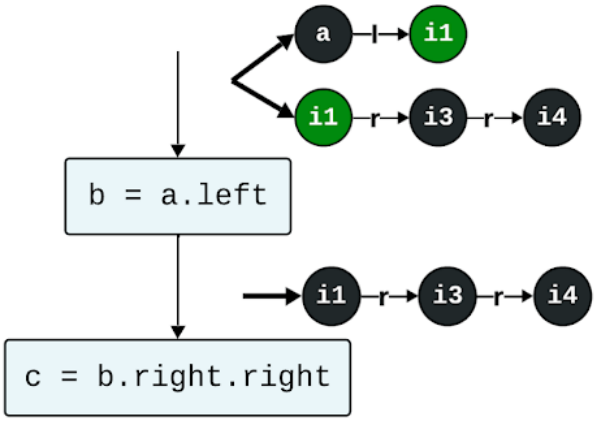


Figure 13: After generating the access graph rooted at `a`, we have two access graphs - one starting from `i1` and the other from `a`. At this stage, we can identify that `i1`, a start node in another access graph, is present as a non-start node elsewhere. This presents an opportunity to merge two access graphs

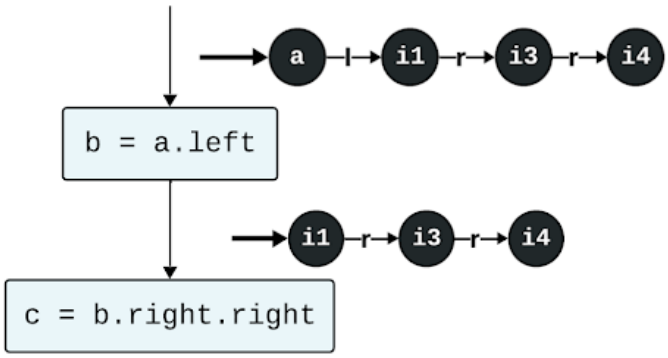


Figure 14: After merging the two graphs, we are left with a single access graph at the entry of the CFG starting from `a`. This is consistent with the result we would obtain using the liveness analysis on the source code

7 Conclusion

Our project allowed us to gain a thorough understanding of the theory behind HRA and also helped us get exposure to frameworks used for program analysis, such as LLVM and VASCO.

We had the opportunity to present our work and this theory through a poster at RISC and PLACID. Several interesting questions were raised during the poster sessions, which highlighted different aspects of the problem and helped us improve our knowledge. One important question raised was how well this analysis performs on benchmarks in terms of the memory saved by nullification and the overhead of performing this analysis. This remains an open question which we hope to answer once the implementation is complete.

The next steps include formalising our modification to the notion of access graphs to make them compatible with SLIM IR, and checking whether the data flow equations and other aspects of the analysis need any modification. Following this, we can work on implementing it in VASCO.

References

[1] U. P. Khedker, A. Sanyal, and A. Karkare, "Heap reference analysis using access graphs," *ACM Trans. Program. Lang. Syst.*, vol. 30, no. 1, p. 1–es, Nov. 2007. [Online]. Available: https://doi.org/10.1145/1290520.1290521